

Git should be installed on the system. If you want to use Google Chrome as the Web browser, make sure Chrome WebDriver is set up properly.

There are two ways in which we can install Splinter.

To install the stable release:

To install the official and bug-free version, run the following command from a terminal:

```
$ [sudo] pip install splinter
```

For installing under-development source-code:

To get Splinter's latest and best features, just run the following given set of commands from a terminal:

```
$ git clone git://github.com/cobrateam/splinter.git
$ cd splinter
$ [sudo] python setup.py install
```

Writing sample code to automate a process using Splinter

As already stated, even a newbie without much knowledge of programming can automate any specific task using Splinter. Let's discover how one can easily make Splinter perform any specific task automatically on a Web application. The credit for the ease of coding actually goes to the different inbuilt functions that Splinter possesses. We just need to incorporate all such built-in functions or library files with the help of a few lines of code. Additionally, we need to apply logic while coding to validate different scenarios from different perspectives. Let's have a look at one of the sample code snippets that has been written for Splinter. Here, we make use of the name and ID values of different elements present on the Web page to identify that specific Web element.

Scenario for sample code: Login to a Facebook account using the user's email ID and password.

```
#imports the Browser library for Splinter
from splinter import Browser

# takes the email address from user as input to login to his/her Facebook account
user_email = raw_input("enter users email address ")
# takes the password from user as input to login to his/her Facebook account
user_pass = raw_input("enter users password ")

# loads the Firefox browser
browser= Browser('firefox')
# stores the URL for Facebook in url variable
url = "https://www.facebook.com/"

#navigates to facebook website and load that in the Firefox browser
```

```
browser.visit(url)
```

```
#checks if Facebook web page is loaded else prints an error
message
```

```
if browser.is_text_present('www.facebook.com'):
```

```
# fills the user's email ID and password in the email and
password field of the facebook login section
```

```
#Inbuilt function browser.fill uses the tag name for Email
and Password input box i.e. email and pass respectively to
identify it
```

```
browser.fill('email', user_email)
```

```
browser.fill('pass', user_pass)
```

```
#selects the login button using its id value present on the
Facebook page to click and log in with the given details
```

```
button = browser.find_by_id('u_o_d')
```

```
button.click()
```

```
else:
```

```
print("Facebook web application NOT FOUND")
```

Some important built-in functions used in Splinter

Table 1 lists some of Splinter's significant built-in functions that can be used while automating any process for a Web application.

Setting up the Splinter development environment

When it comes to programming in Splinter, we have already seen that it's easier than other open source Web application testing tools. But we need to set up a development environment for it, wherein we can easily code or automate a specific process using Splinter. This is not a tough task. We just need to make sure that we have some basic development tools, library files and a few add-on dependencies on our machine, which will ultimately help us code in an easier and better way. We can get the required tools and set up the entire environment using just a few commands.

Lets' have a look at the different development tools required to set up the environment.

Basic development tools: If you are using the Mac OS, install the Xcode tool. It can be downloaded from the Mac Application Store (on the Mac OS X Lion) or even from the Apple website.

If you are using a Linux computer, install some of the basic development libraries and the headers. On Ubuntu, you can easily install all of these using the *apt-get* command. Given below is the command used for this purpose.

```
$ [sudo] apt-get install build-essential python-dev libxml2-dev
libxslt1-dev
```

Pip and virtualenv: First of all, we need to make sure that we have Pip installed in our system, with which we manage all the Splinter development dependencies. It lets us program our task and makes the system perform any activity